

Fake Job Posting Detection

A Machine Learning Approach to Identifying Fraudulent Job Postings

Project Report

Submitted by: Vaishnavi Sharma

3Skill AI/ML Internship Program

June 2026- August 2026

DECLARATION

I hereby declare that the project entitled "**Fake Job Posting Detection Using Machine Learning**" submitted as part of the **3Skill AI & Machine Learning Internship Program** is my original work. The project has been developed by me under the guidance and learning provided during the internship.

This report has been prepared solely for the purpose of internship evaluation and certification. The work presented in this project has not been submitted, either in whole or in part, to any other institution, organization, or university for the award of any degree, diploma, certificate, or any other academic qualification.

All information, datasets, software libraries, and resources used in this project have been appropriately acknowledged wherever required. Every effort has been made to ensure the correctness and authenticity of the work presented in this report.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **3Skill** for providing me with the opportunity to participate in the **AI & Machine Learning Internship Program**. This internship provided me with valuable practical exposure to Machine Learning concepts and enabled me to apply theoretical knowledge to a real-world problem through this project.

I am thankful to all the mentors, trainers, and the entire 3Skill team for their continuous guidance, learning resources, and support throughout the internship. Their valuable suggestions and encouragement helped me understand various stages of a Machine Learning project, including data preprocessing, exploratory data analysis, feature engineering, model development, model evaluation, hyperparameter tuning, and deployment of a prediction system.

I would also like to express my gratitude to my college, faculty members, and everyone who directly or indirectly supported me during the completion of this project.

Finally, I am grateful for this opportunity to strengthen my technical knowledge and practical skills in Python, Data Analysis, and Machine Learning. This project has significantly enhanced my understanding of solving real-world problems using data-driven approaches and has motivated me to continue learning and exploring the field of Artificial Intelligence.

ABSTRACT

The rapid growth of online recruitment platforms has made job searching easier for applicants worldwide. However, it has also led to an increase in fraudulent job postings that deceive job seekers through fake employment opportunities, misleading information, and financial scams. Detecting such fraudulent job advertisements manually is difficult due to the large volume of job postings published every day. Therefore, an automated detection system based on Machine Learning can assist in identifying suspicious job postings efficiently.

The objective of this project is to develop a Machine Learning-based system capable of classifying job postings as **Fraudulent** or **Legitimate** by analyzing textual and structured information available in the dataset. The project begins with Exploratory Data Analysis (EDA), where the dataset is examined to understand its structure, identify missing values, analyze feature distributions, and study the target variable. Appropriate preprocessing techniques are then applied, including handling missing values, combining relevant text features, cleaning textual data by removing punctuation and stop words, and converting text into numerical form using **TF-IDF Vectorization**. An additional experiment is also conducted using **CountVectorizer** to compare the impact of different text vectorization techniques on model performance.

Several Machine Learning classification algorithms, namely **Logistic Regression**, **Naive Bayes**, **Decision Tree**, and **Random Forest**, are trained and evaluated using Accuracy, Precision, Recall, and F1 Score. Furthermore, **GridSearchCV** is used to perform hyperparameter tuning on Logistic Regression in order to improve its performance. The project also includes a functional prediction system capable of classifying unseen job postings and a model interpretation section to understand how textual features influence the prediction process.

The results demonstrate that Machine Learning can effectively identify fraudulent job postings with high accuracy. The developed system provides a practical approach for assisting online recruitment platforms and job seekers in detecting potentially fraudulent job advertisements while reducing the risk of employment-related scams.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
1.	Introduction	6
2.	Problem Statement	7
3.	Dataset Description	8
4.	Exploratory Data Analysis (EDA)	12
5.	Data Preprocessing	16
6.	Feature Engineering	20
7.	Machine Learning Models	24
8.	Model Training and Evaluation	27
9.	Hyperparameter Tuning	30
10.	Prediction System	32
11.	Model Interpretation	36
12.	Preprocessing Technique Comparison	39
13.	Conclusion	42
14.	Limitations	43
15.	Future Scope	44
16.	References	45

INTRODUCTION

The rapid growth of the internet and online recruitment platforms has transformed the hiring process by making job searching faster and more accessible. However, this growth has also led to an increase in fraudulent job postings that deceive applicants through fake employment opportunities, misleading information, and financial scams. These fake advertisements often promise attractive salaries, flexible working hours, or quick hiring processes to collect personal or financial information from job seekers.

Manually identifying fraudulent job postings is difficult because genuine and fake advertisements often appear similar. As thousands of job advertisements are published daily, manual verification becomes impractical. Therefore, automated detection systems are needed to improve the security and reliability of online recruitment platforms.

Machine Learning provides an effective solution by learning patterns from historical data and classifying job postings as **Legitimate** or **Fraudulent**. By analyzing textual information such as job titles, company profiles, job descriptions, requirements, and benefits, along with structured job-related features, machine learning models can identify characteristics commonly associated with fraudulent advertisements.

In this project, a Machine Learning-based Fake Job Posting Detection system was developed using Python and various data science libraries. The project began with **Exploratory Data Analysis (EDA)** to understand the dataset and identify missing values. The textual data was then preprocessed by handling missing values, cleaning the text, and converting it into numerical features using **TF-IDF Vectorization**. In addition, **CountVectorizer** was implemented to compare the impact of different text vectorization techniques on model performance.

Several classification algorithms, including **Logistic Regression, Naive Bayes, Decision Tree, and Random Forest**, were trained and evaluated using **Accuracy, Precision, Recall, and F1 Score**. Logistic Regression was further improved using **GridSearchCV** for hyperparameter tuning. A prediction system was also developed to classify unseen job postings, and model interpretation was performed to understand the important features influencing the predictions.

The results demonstrate that Machine Learning can effectively detect fraudulent job postings and assist online recruitment platforms in reducing employment scams. This project highlights the practical application of Machine Learning and Natural Language Processing in improving the security, reliability, and trustworthiness of online recruitment systems.

PROBLEM STATEMENT

With the increasing popularity of online recruitment platforms, job seekers have access to thousands of employment opportunities across different industries. However, the rapid growth of online job portals has also resulted in a significant rise in fraudulent job postings. These fake advertisements are designed to deceive applicants by offering unrealistic salaries, attractive benefits, or quick hiring processes while attempting to collect personal information, financial details, or money through fraudulent means.

Identifying fake job postings manually is a difficult and time-consuming process because genuine and fraudulent advertisements often share similar formats and language. As the number of online job postings continues to grow, manual verification becomes impractical for both recruitment platforms and job seekers. Consequently, many fraudulent job advertisements remain undetected, exposing applicants to scams, identity theft, and financial losses.

To address this issue, there is a need for an intelligent and automated system capable of distinguishing between legitimate and fraudulent job postings. Machine Learning provides an effective approach by learning patterns from previously labeled data and applying that knowledge to classify new and unseen job advertisements.

In this project, a Machine Learning-based Fake Job Posting Detection system is developed using textual and structured information available in the dataset. The system performs data preprocessing, exploratory data analysis, feature engineering, text vectorization, model training, hyperparameter tuning, and performance evaluation before predicting whether a job posting is **Fraudulent** or **Legitimate**.

The proposed solution aims to assist job seekers and online recruitment platforms by providing a reliable and efficient method for identifying suspicious job advertisements, thereby improving user safety and enhancing trust in online recruitment systems.

DATASET DESCRIPTION

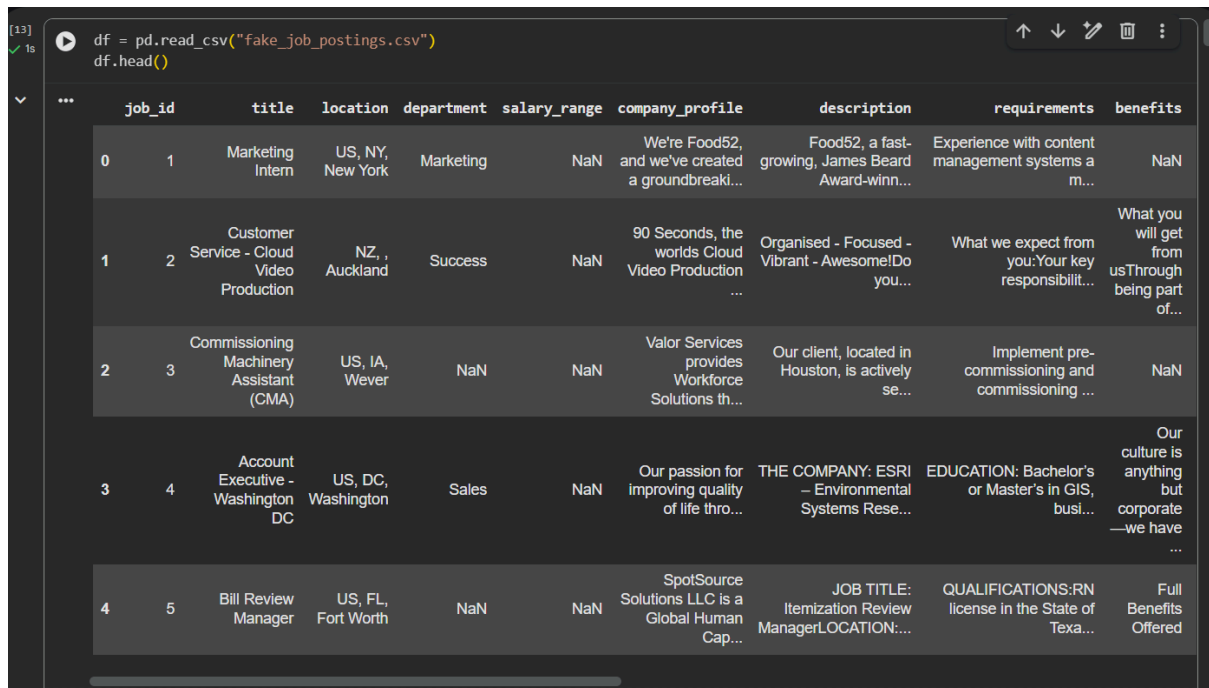
3.1 Dataset Overview

The dataset used in this project is the **Fake Job Posting Prediction Dataset**, obtained from **Kaggle**. The dataset contains job advertisements collected from online recruitment platforms. Each record represents a single job posting and includes both textual information (such as job descriptions and company profiles) and structured information (such as employment type, required education, and industry).

The objective of the dataset is to classify each job posting into one of two categories:

- **0 – Legitimate Job Posting**
- **1 – Fraudulent Job Posting**

This dataset is well suited for Natural Language Processing (NLP) and Machine Learning classification tasks because it combines structured categorical features with large amounts of textual information.



The screenshot shows a Jupyter Notebook interface with a code cell and its output. The code cell contains the following Python code:

```
[13]: df = pd.read_csv("fake_job_postings.csv")
df.head()
```

The output is a pandas DataFrame with the following columns: job_id, title, location, department, salary_range, company_profile, description, requirements, and benefits. The DataFrame contains 5 rows of data, each representing a job posting.

	job_id	title	location	department	salary_range	company_profile	description	requirements	benefits
0	1	Marketing Intern	US, NY, New York	Marketing	NaN	We're Food52, and we've created a groundbreaki...	Food52, a fast-growing, James Beard Award-winn...	Experience with content management systems a m...	NaN
1	2	Customer Service - Cloud Video Production	NZ, , Auckland	Success	NaN	90 Seconds, the worlds Cloud Video Production ...	Organised - Focused - Vibrant - Awesome!Do you...	What we expect from you:Your key responsibilit...	What you will get from usThrough being part of...
2	3	Commissioning Machinery Assistant (CMA)	US, IA, Wever	NaN	NaN	Valor Services provides Workforce Solutions th...	Our client, located in Houston, is actively se...	Implementing pre-commissioning and commissioning ...	NaN
3	4	Account Executive - Washington DC	US, DC, Washington	Sales	NaN	Our passion for improving quality of life thro...	THE COMPANY: ESRI – Environmental Systems Rese...	EDUCATION: Bachelor's or Master's in GIS, busi...	Our culture is anything but corporate—we have ...
4	5	Bill Review Manager	US, FL, Fort Worth	NaN	NaN	SpotSource Solutions LLC is a Global Human Cap...	JOB TITLE: Itemization Review ManagerLOCATION:...	QUALIFICATIONS:RN license in the State of Texa...	Full Benefits Offered

3.2 Dataset Features

The following are the main attributes used in the dataset:

Feature Name	Description
--------------	-------------

title	Job title of the advertised position.
company_profile	Information describing the company offering the job.
description	Detailed description of the job responsibilities and duties.
requirements	Skills, qualifications, and experience required for the position.
benefits	Benefits and additional facilities offered to employees.
employment_type	Type of employment such as Full-time, Part-time, Contract, etc.
required_experience	Experience level required for the job.
required_education	Educational qualification expected from applicants.
industry	Industry to which the job belongs.
function	Functional area or department of the job.
telecommuting	Indicates whether remote work is available.
has_company_logo	Indicates whether the company logo is provided.
has_questions	Indicates whether screening questions are included.
fraudulent	Target variable indicating whether the job posting is fake (1) or legitimate (0).

3.3 Exploratory Data Analysis

Before developing the Machine Learning models, the dataset was explored to understand its overall structure and quality. The following analyses were performed:

- Displayed the first few records of the dataset using `df.head()`.
- Examined the dataset dimensions using `df.shape`.
- Checked data types and non-null values using `df.info()`.
- Displayed column names using `df.columns`.
- Generated descriptive statistics using `df.describe()`.
- Identified missing values using `df.isnull().sum()`.
- Analyzed the distribution of fraudulent and legitimate job postings.
- Visualized missing values using a heatmap.
- Examined the relationship between company profiles and fraudulent postings.
- Calculated the average description length for both genuine and fraudulent job postings.

These analyses helped identify missing values, understand feature distributions, and determine the preprocessing steps required before model training.

```
print(df.shape)
df.info()

... (17880, 18)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17880 entries, 0 to 17879
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   job_id                17880 non-null  int64
1   title                 17880 non-null  object
2   location              17534 non-null  object
3   department            6333 non-null   object
4   salary_range          2868 non-null   object
5   company_profile       14572 non-null  object
6   description           17879 non-null  object
7   requirements          15184 non-null  object
8   benefits              10668 non-null  object
9   telecommuting         17880 non-null  int64
10  has_company_logo      17880 non-null  int64
11  has_questions         17880 non-null  int64
12  employment_type       14409 non-null  object
13  required_experience    10830 non-null  object
14  required_education    9775 non-null   object
15  industry              12977 non-null  object
16  function              11425 non-null  object
17  fraudulent            17880 non-null  int64
dtypes: int64(5), object(13)
memory usage: 2.5+ MB
```

Missing Value Summary Table

```
missing_data = pd.DataFrame({
    "Missing Values": df.isnull().sum(),
    "Percentage": round((df.isnull().sum()/len(df))*100,2)
})

missing_data = missing_data.sort_values(
    by="Percentage",
    ascending=False
)

missing_data
```

	Missing Values	Percentage
salary_range	15012	83.96
department	11547	64.58
required_education	8105	45.33
benefits	7212	40.34
required_experience	7050	39.43
function	6455	36.10
industry	4903	27.42
employment_type	3471	19.41
company_profile	3308	18.50

requirements	2696	15.08
location	346	1.94
description	1	0.01
title	0	0.00
job_id	0	0.00
telecommuting	0	0.00
has_questions	0	0.00
has_company_logo	0	0.00
fraudulent	0	0.00

Next steps: [Generate code with missing_data](#) [New interactive sheet](#)

3.4 Dataset Preparation

After the initial analysis, the dataset was prepared for Machine Learning by:

- Handling missing values in relevant columns.
- Creating additional features such as company profile availability and description length.
- Combining important textual columns into a single text feature.
- Cleaning the text by converting it to lowercase, removing punctuation, removing stop words, and applying lemmatization.
- Converting the cleaned text into numerical features using TF-IDF Vectorization.
- Performing an additional experiment using CountVectorizer to compare different text vectorization techniques.

These preprocessing steps improved the quality of the input data and prepared it for effective model training and evaluation.

EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis (EDA) is an important step in every Machine Learning project. It helps in understanding the structure of the dataset, identifying missing values, analyzing the distribution of features, and discovering relationships between variables before building predictive models. Proper EDA improves data quality and assists in selecting appropriate preprocessing techniques.

In this project, several exploratory analyses were performed to gain insights into the Fake Job Posting dataset.

4.1 Dataset Exploration

The dataset was initially explored using various Pandas functions to understand its structure.

The following operations were performed:

- `df.head()` was used to display the first five records of the dataset.
- `df.shape` was used to determine the number of rows and columns.
- `df.info()` provided information about data types and non-null values.
- `df.columns` displayed all available feature names.
- `df.describe()` generated descriptive statistics for numerical attributes.

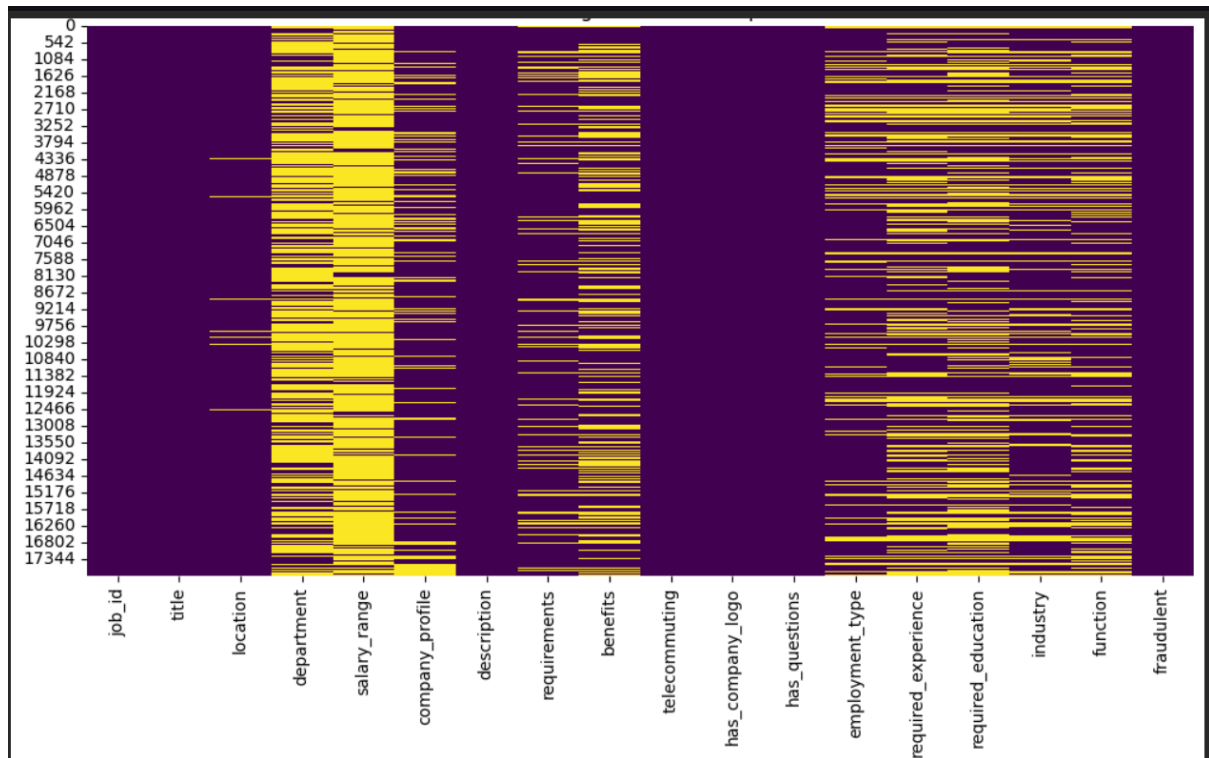
These functions provided an overall understanding of the dataset before preprocessing.

4.2 Missing Value Analysis

Missing values were identified using the following approach:

- Calculated the number of missing values in each column.
- Sorted the missing values in descending order to identify the columns requiring preprocessing.
- Visualized missing values using a heatmap generated with the Seaborn library.

This analysis helped determine which features required handling before model training. Missing textual fields were appropriately processed to avoid data loss and improve model performance.



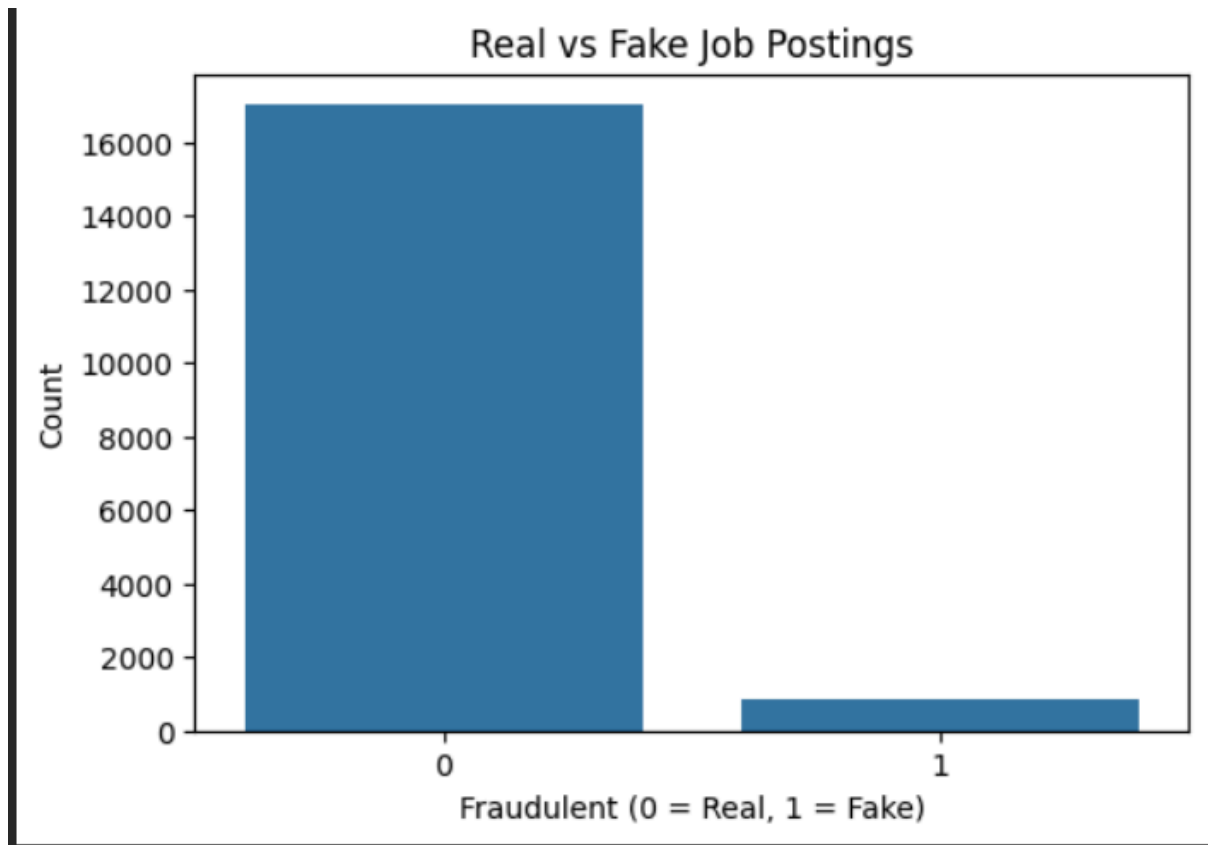
4.3 Target Variable Analysis

The target variable (**fraudulent**) was analyzed to understand the distribution of legitimate and fake job postings.

The following analyses were performed:

- Count of legitimate and fraudulent job postings.
- Percentage distribution of each class.

Understanding the class distribution helped evaluate whether the dataset was balanced and highlighted the importance of using multiple evaluation metrics such as Precision, Recall, and F1 Score instead of relying solely on Accuracy.



4.4 Feature Analysis

Additional exploratory analyses were performed to understand important dataset characteristics.

These included:

- Creating a feature indicating whether a company profile was available.
- Studying the relationship between the presence of a company profile and fraudulent job postings.
- Calculating the length of each job description.
- Comparing the average description length for legitimate and fraudulent postings.

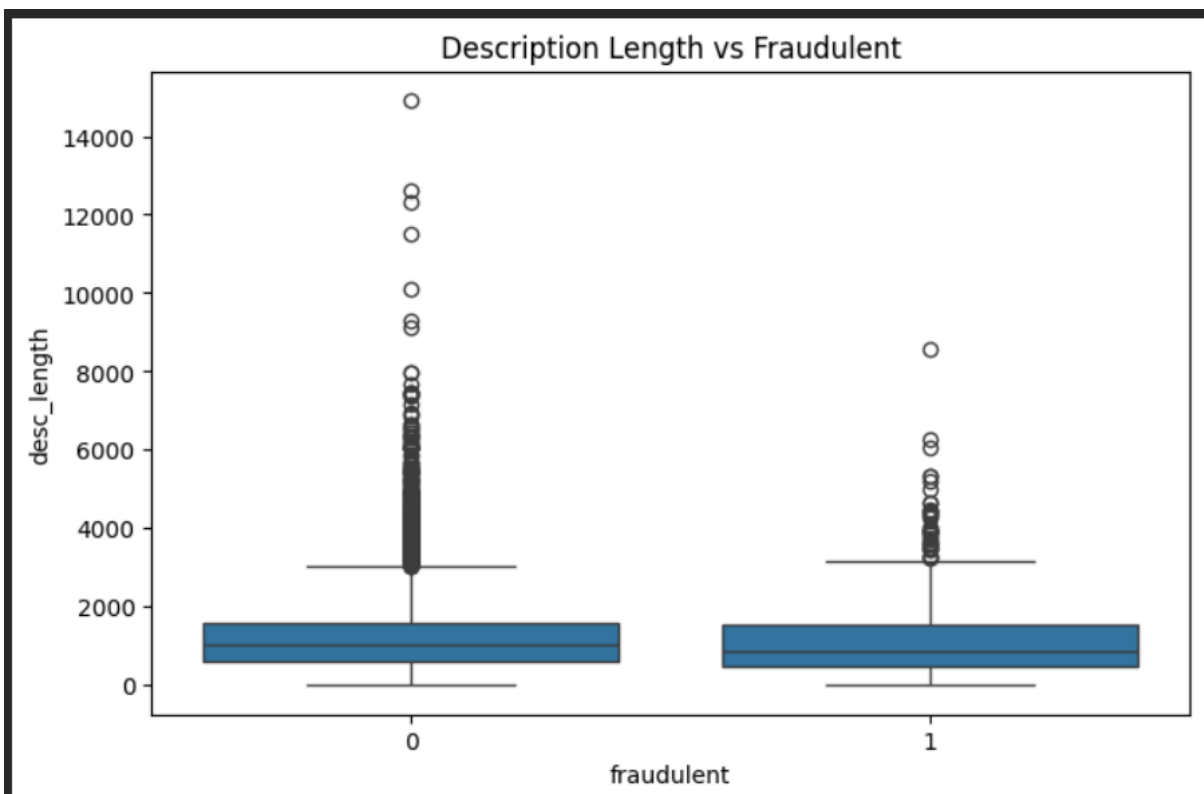
These analyses provided useful insights into how different attributes might contribute to the prediction of fraudulent job advertisements.

Missing Company Profile Analysis

```
df["has_company_profile"] = df["company_profile"].notnull().astype(int)

pd.crosstab(
    df["has_company_profile"],
    df["fraudulent"],
    normalize="index"
)
```

has_company_profile	fraudulent	
	0	1
0	0.822551	0.177449
1	0.980854	0.019146



DATA PREPROCESSING

Data preprocessing is one of the most important stages in a Machine Learning project. Raw data often contains missing values, inconsistent formatting, unnecessary symbols, and unstructured textual information that can negatively affect the performance of machine learning models. Therefore, before training the models, the dataset was carefully cleaned and transformed into a suitable format for analysis.

The preprocessing steps performed in this project are described below.

5.1 Removal of Unnecessary Features

The dataset originally contained several columns that were not useful for predicting whether a job posting was fraudulent. Features that did not contribute meaningful information to the classification process were removed to simplify the dataset and improve model efficiency.

Only the most relevant textual and structured features were retained for further analysis and model training.

5.2 Handling Missing Values

Several columns contained missing values. Appropriate preprocessing techniques were applied to ensure that missing data did not negatively affect the machine learning models.

The following strategies were used:

- Missing values in textual columns were replaced with empty strings.
- Structured features were preserved wherever possible.
- Additional features created during exploratory analysis were retained for further processing.

This approach ensured that all records could be utilized without unnecessarily removing valuable information.

5.3 Text Feature Combination

Since important information was distributed across multiple textual columns, these columns were combined into a single text feature before further processing.

The following columns were merged:

- Job Title
- Company Profile
- Job Description

- Requirements
- Benefits

Combining these attributes allowed the model to learn from all available textual information instead of relying on a single field.

```
df_clean["text"] = (  
    df_clean["title"] + " " +  
    df_clean["company_profile"] + " " +  
    df_clean["description"] + " " +  
    df_clean["requirements"] + " " +  
    df_clean["benefits"]  
)
```

5.4 Text Cleaning

The combined text was cleaned using Natural Language Processing (NLP) techniques to improve the quality of textual data.

The following preprocessing operations were performed:

- Conversion of all text to lowercase.
- Removal of punctuation marks.
- Removal of special characters.
- Removal of English stop words.
- Lemmatization to convert words into their root forms.
- Removal of extra white spaces.

These cleaning steps reduced noise in the dataset and improved the quality of features used for model training.

```
df_clean["text"].head()
```

	text
0	Marketing Intern We're Food52, and we've creat...
1	Customer Service - Cloud Video Production 90 S...
2	Commissioning Machinery Assistant (CMA) Valor ...
3	Account Executive - Washington DC Our passion ...
4	Bill Review Manager SpotSource Solutions LLC i...

dtype: object

5.5 Text Vectorization

Machine Learning algorithms cannot process raw text directly. Therefore, the cleaned textual data was converted into numerical feature vectors.

The primary vectorization technique used in this project was:

- **TF-IDF (Term Frequency–Inverse Document Frequency)**

TF-IDF assigns importance scores to words based on how frequently they appear within a document relative to the entire dataset. This helps reduce the influence of very common words while emphasizing more informative terms.

To analyze the impact of preprocessing techniques, an additional experiment was conducted using **CountVectorizer**. The performance of both approaches was later compared using standard evaluation metrics.

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(
    stop_words="english",
    max_features=10000,
    ngram_range=(1,2),
    min_df=2
)

X_train_tfidf = tfidf.fit_transform(X_train)

X_test_tfidf = tfidf.transform(X_test)
```

5.6 Data Splitting

After preprocessing and vectorization, the dataset was divided into training and testing subsets.

- **Training Set:** Used to train the machine learning models.
- **Testing Set:** Used to evaluate model performance on previously unseen data.

This separation ensured that the evaluation accurately reflected the model's ability to generalize to new job postings.

```
▶ from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training Samples:", X_train.shape)
print("Testing Samples :", X_test.shape)

... Training Samples: (14304,)
Testing Samples : (3576,)
```

Summary

The preprocessing stage transformed the raw dataset into a clean and structured format suitable for Machine Learning. Missing values were handled, important textual fields were combined, text was cleaned using NLP techniques, and numerical feature vectors were generated using TF-IDF. An additional comparison using CountVectorizer was also performed to study the impact of different text vectorization techniques on model performance.

FEATURE ENGINEERING

Feature engineering is the process of creating meaningful input features from raw data to improve the performance of machine learning models. Well-designed features help the model learn useful patterns and relationships more effectively. In this project, both structured and textual features were prepared before training the classification models.

6.1 Company Profile Indicator

A new feature named **has_company_profile** was created to indicate whether a company profile was available for a job posting.

The values were assigned as follows:

- **1** – Company profile is available.
- **0** – Company profile is missing.

This feature was created to analyze whether the presence of a company profile has any relationship with fraudulent job postings. During the exploratory data analysis, a cross-tabulation was performed to compare this feature with the target variable (**fraudulent**), providing useful insights into the credibility of job postings.

Missing Company Profile Analysis

```
df["has_company_profile"] = df["company_profile"].notnull().astype(int)

pd.crosstab(
    df["has_company_profile"],
    df["fraudulent"],
    normalize="index"
)
```

	fraudulent	
	0	1
has_company_profile		
0	0.822551	0.177449
1	0.980854	0.019146

6.2 Job Description Length

Another feature named **desc_length** was created by calculating the total number of characters present in the job description.

This feature was used to compare the average description length of legitimate and fraudulent job postings. The analysis helped identify whether fraudulent advertisements generally contained shorter or longer descriptions compared to genuine postings.

Although this feature was mainly used for exploratory analysis, it provided additional understanding of the dataset characteristics.

Description Length

```
df["description"] = df["description"].fillna("")  
df["desc_length"] = df["description"].apply(len)  
df.groupby("fraudulent")["desc_length"].mean()
```

```
...      desc_length  
fraudulent  
0      1221.219701  
1      1154.834873  
dtype: float64
```

6.3 Combined Text Feature

The dataset contained important textual information distributed across multiple columns. Instead of processing each column separately, they were combined into a single text feature to provide the model with more complete contextual information.

The following textual columns were merged:

- Title
- Company Profile
- Description
- Requirements
- Benefits

Combining these fields enabled the machine learning models to learn from all available textual information within each job posting.

```
df_clean["text"].head()
```

	text
0	Marketing Intern We're Food52, and we've creat...
1	Customer Service - Cloud Video Production 90 S...
2	Commissioning Machinery Assistant (CMA) Valor ...
3	Account Executive - Washington DC Our passion ...
4	Bill Review Manager SpotSource Solutions LLC i...

dtype: object

6.4 Clean Text Feature

After combining the textual columns, a new feature named **clean_text** was generated through text preprocessing.

The preprocessing steps included:

- Converting all text to lowercase.
- Removing punctuation marks.
- Removing special characters.
- Removing English stop words.
- Applying lemmatization.
- Removing unnecessary white spaces.

The **clean_text** feature served as the final textual input for text vectorization.

```
df_clean[["text", "clean_text"]].head()
```

	text	clean_text
0	Marketing Intern We're Food52, and we've creat...	marketing intern food created groundbreaking a...
1	Customer Service - Cloud Video Production 90 S...	customer service cloud video production second...
2	Commissioning Machinery Assistant (CMA) Valor ...	commissioning machinery assistant cma valor se...
3	Account Executive - Washington DC Our passion ...	account executive washington dc passion improv...
4	Bill Review Manager SpotSource Solutions LLC i...	bill review manager spotsource solution llc gl...

6.5 Numerical Feature Representation

Since Machine Learning algorithms require numerical input, the cleaned text was transformed into numerical vectors.

The primary feature representation technique used in this project was **TF-IDF (Term Frequency–Inverse Document Frequency)**. This technique assigns higher importance to informative words while reducing the influence of frequently occurring words.

Additionally, **CountVectorizer** was used as an alternative text representation technique to compare its impact on model performance. This comparison helped evaluate how different preprocessing approaches affected classification accuracy and other evaluation metrics.

Importance of Feature Engineering

The engineered features significantly improved the quality of the input data by:

- Combining important textual information into a unified feature.
- Creating additional informative variables.
- Reducing noise through text cleaning.
- Converting unstructured text into numerical representations suitable for Machine Learning.

These engineered features formed the foundation for training and evaluating the classification models developed in this project.

MACHINE LEARNING MODELS

After completing data preprocessing and feature engineering, several Machine Learning classification algorithms were implemented to identify whether a job posting was **Fraudulent** or **Legitimate**. Using multiple algorithms allowed a comparative analysis of their performance and helped determine the most suitable model for the Fake Job Posting Detection task.

The models were trained using the training dataset and evaluated on the testing dataset using standard classification metrics such as **Accuracy, Precision, Recall, and F1 Score**.

7.1 Logistic Regression

Logistic Regression is a supervised machine learning algorithm commonly used for binary classification problems. It estimates the probability that an input belongs to one of two classes using the logistic (sigmoid) function.

In this project, Logistic Regression was trained on the TF-IDF feature vectors generated from the cleaned textual data. Due to its simplicity, efficiency, and ability to perform well on high-dimensional text data, it served as one of the primary baseline models.

Furthermore, the performance of Logistic Regression was improved through **hyperparameter tuning using GridSearchCV**, making it the final model used in the prediction system.

```
lr_model = LogisticRegression(max_iter=1000)

lr_model.fit(x_train_tfidf, y_train)

lr_predictions = lr_model.predict(x_test_tfidf)
```

7.2 Naive Bayes

Naive Bayes is a probabilistic classification algorithm based on Bayes' Theorem. It assumes that the input features are conditionally independent of one another, which makes it computationally efficient for text classification tasks.

Since the project primarily involves textual data, Naive Bayes was selected as one of the baseline models to evaluate its effectiveness in detecting fraudulent job postings.

Multinomial Naive Bayes

```
nb_model = MultinomialNB()

nb_model.fit(x_train_tfidf, y_train)

nb_predictions = nb_model.predict(x_test_tfidf)
```

7.3 Decision Tree

Decision Tree is a supervised learning algorithm that classifies data by creating a tree-like structure of decision rules. It recursively splits the data based on feature values until the final classification is obtained.

The Decision Tree model was included in this project to compare a rule-based learning approach with probabilistic and linear classification methods.

Decision Tree

```
dt_model = DecisionTreeClassifier(
    random_state=42
)

dt_model.fit(
    x_train_tfidf,
    y_train
)

dt_predictions = dt_model.predict(
    x_test_tfidf
)
```

7.4 Random Forest

Random Forest is an ensemble learning algorithm that combines multiple Decision Trees to improve prediction accuracy and reduce overfitting.

Each decision tree is trained using a random subset of the training data and features. The final prediction is determined by majority voting among all trees in the forest.

Random Forest demonstrated strong performance during model comparison and served as one of the best-performing baseline models before hyperparameter tuning of Logistic Regression.

Random Forest

```
rf_model = RandomForestClassifier(  
    n_estimators=200,  
    random_state=42  
)  
  
rf_model.fit(  
    x_train_tfidf,  
    y_train  
)  
  
rf_predictions = rf_model.predict(  
    x_test_tfidf  
)
```

7.5 Model Selection Strategy

Instead of selecting a single algorithm directly, multiple Machine Learning models were trained and evaluated using identical training and testing datasets. Their performances were compared using multiple evaluation metrics to understand their strengths and weaknesses.

The comparison was based on:

- Accuracy
- Precision
- Recall
- F1 Score

Using multiple evaluation metrics provided a more reliable assessment than relying solely on accuracy, particularly because fraudulent job detection requires correctly identifying fraudulent postings while minimizing false predictions.

Summary

The implementation of multiple Machine Learning algorithms enabled a comprehensive comparison of different classification techniques. Logistic Regression, Naive Bayes, Decision Tree, and Random Forest each demonstrated unique characteristics during evaluation. The comparative analysis and subsequent hyperparameter tuning helped identify the most appropriate model for developing the final Fake Job Posting Detection system.

MODEL TRAINING AND EVALUATION

After completing data preprocessing and feature engineering, the prepared dataset was divided into training and testing subsets using the **train-test split** technique. The training dataset was used to train the machine learning models, while the testing dataset was reserved for evaluating their performance on previously unseen data. This approach ensured that the evaluation reflected the model's ability to generalize to new job postings rather than memorizing the training data.

Four machine learning classification algorithms were trained in this project:

- Logistic Regression
- Naive Bayes
- Decision Tree
- Random Forest

Each model was trained using the same training dataset to ensure a fair comparison. After training, predictions were generated on the testing dataset and evaluated using multiple performance metrics.

8.1 Evaluation Metrics

To obtain a comprehensive assessment of model performance, the following evaluation metrics were used:

Accuracy

Accuracy measures the proportion of correctly classified job postings out of the total number of job postings.

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Although accuracy provides an overall measure of correctness, it may not be sufficient when dealing with fraud detection problems. Therefore, additional evaluation metrics were considered.

Precision

Precision measures the proportion of job postings predicted as fraudulent that were actually fraudulent.

$$\text{Precision} = TP / (TP + FP)$$

A high precision value indicates that the model generates fewer false fraud predictions.

Recall

Recall measures the proportion of actual fraudulent job postings that were successfully identified by the model.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Since the primary objective is to detect fraudulent job advertisements, recall is an important metric because it indicates how effectively the model identifies fake job postings.

F1 Score

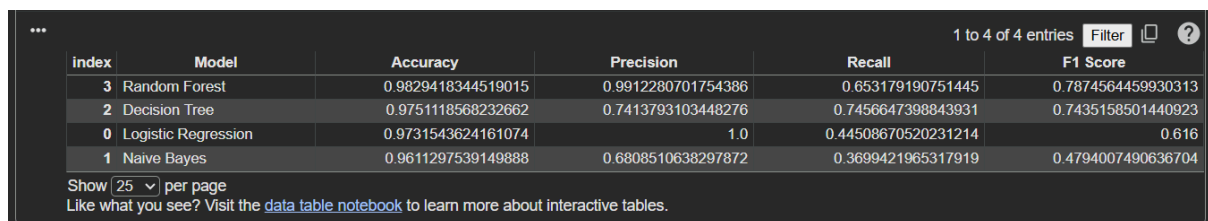
The F1 Score is the harmonic mean of Precision and Recall.

$$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

The F1 Score provides a balanced measure of model performance and is particularly useful for fraud detection tasks where both Precision and Recall are important.

8.2 Model Comparison

The performance of all four machine learning models was compared using the evaluation metrics described above.



The screenshot shows an interactive table with 5 columns: Index, Model, Accuracy, Precision, Recall, and F1 Score. There are 4 rows of data. The table is titled '1 to 4 of 4 entries' and includes a 'Filter' button and a help icon. Below the table, there is a 'Show / 25 per page' dropdown and a link to 'data table notebook'.

Index	Model	Accuracy	Precision	Recall	F1 Score
3	Random Forest	0.9829418344519015	0.9912280701754386	0.653179190751445	0.7874564459930313
2	Decision Tree	0.9751118568232662	0.7413793103448276	0.7456647398843931	0.7435158501440923
0	Logistic Regression	0.9731543624161074	1.0	0.44508670520231214	0.616
1	Naive Bayes	0.9611297539149888	0.6808510638297872	0.3699421965317919	0.4794007490638704

8.3 Model Analysis

The experimental results demonstrate that each machine learning algorithm has its own strengths and limitations.

- Logistic Regression produced reliable baseline performance for text classification.
- Naive Bayes performed efficiently but achieved comparatively lower overall performance than the other models.
- Decision Tree successfully learned decision rules from the training data but was more prone to overfitting.
- Random Forest achieved the strongest performance among the baseline models due to its ensemble learning approach and better generalization capability.

To further improve classification performance, Logistic Regression was optimized using **GridSearchCV**, which is discussed in the next chapter.

Logistic Regression Results

```
▶ print("Accuracy :", accuracy_score(y_test, lr_predictions))
  print("Precision :", precision_score(y_test, lr_predictions))
  print("Recall :", recall_score(y_test, lr_predictions))
  print("F1 Score :", f1_score(y_test, lr_predictions))
```

```
... Accuracy : 0.9731543624161074
  Precision : 1.0
  Recall : 0.44508670520231214
  F1 Score : 0.616
```

Naive Bayes Results

```
▶ print("Accuracy :", accuracy_score(y_test, nb_predictions))
  print("Precision :", precision_score(y_test, nb_predictions))
  print("Recall :", recall_score(y_test, nb_predictions))
  print("F1 Score :", f1_score(y_test, nb_predictions))
```

```
... Accuracy : 0.9611297539149888
  Precision : 0.6808510638297872
  Recall : 0.3699421965317919
  F1 Score : 0.4794007490636704
```

Decision Tree Results

```
▶ print("Accuracy :", accuracy_score(y_test, dt_predictions))
  print("Precision :", precision_score(y_test, dt_predictions))
  print("Recall :", recall_score(y_test, dt_predictions))
  print("F1 Score :", f1_score(y_test, dt_predictions))
```

```
... Accuracy : 0.9751118568232662
  Precision : 0.7413793103448276
  Recall : 0.7456647398843931
  F1 Score : 0.7435158501440923
```

Random Forest Results

```
▶ print("Accuracy :", accuracy_score(y_test, rf_predictions))
  print("Precision :", precision_score(y_test, rf_predictions))
  print("Recall :", recall_score(y_test, rf_predictions))
  print("F1 Score :", f1_score(y_test, rf_predictions))
```

```
... Accuracy : 0.9829418344519015
  Precision : 0.9912280701754386
  Recall : 0.653179190751445
  F1 Score : 0.7874564459930313
```

Summary

Multiple machine learning algorithms were successfully trained and evaluated using the same dataset and evaluation criteria. Comparing multiple models provided a better understanding of their strengths and weaknesses and established a strong baseline before applying hyperparameter tuning. The evaluation process demonstrated the importance of considering multiple metrics rather than relying solely on accuracy when solving a fraud detection problem.

HYPERPARAMETER TUNING

Machine Learning models contain parameters that control their learning process. These parameters, known as **hyperparameters**, are set before training begins and can significantly influence the performance of a model. Selecting appropriate hyperparameter values helps improve prediction accuracy, reduce overfitting, and enhance the model's ability to generalize to unseen data.

In this project, **Logistic Regression** was selected for hyperparameter tuning because it demonstrated strong baseline performance and is widely used for text classification problems.

9.1 GridSearchCV

To optimize the Logistic Regression model, **GridSearchCV** from the Scikit-learn library was used.

GridSearchCV is a systematic hyperparameter optimization technique that evaluates multiple combinations of parameter values using **cross-validation**. Instead of manually selecting parameter values, GridSearchCV automatically identifies the combination that produces the best overall performance.

The major advantages of GridSearchCV include:

- Automatic search for the best hyperparameter combination.
- Use of cross-validation to produce reliable evaluation results.
- Reduction in manual trial-and-error during model optimization.
- Improved model performance and generalization.

```
param_grid = {  
    "C": [0.1, 1, 10],  
    "solver": ["liblinear", "lbfgs"]  
}
```

9.2 Hyperparameters Used

The following Logistic Regression hyperparameters were explored during the tuning process:

- **C**: Controls the regularization strength of the model.
- **Penalty**: Specifies the regularization method used by the algorithm.
- **Solver**: Determines the optimization algorithm used to train the Logistic Regression model.

Different combinations of these parameters were evaluated automatically by GridSearchCV, and the best-performing configuration was selected.

Best Parameters

```
print("Best Parameters:")
print(grid_search.best_params_)

print("\nBest Cross Validation F1 Score:")
print(round(grid_search.best_score_, 4))

... Best Parameters:
{'C': 10, 'solver': 'liblinear'}

Best Cross Validation F1 Score:
0.7854
```

9.3 Cross-Validation

GridSearchCV performs **cross-validation** during model training.

Instead of evaluating the model using only one train-test split, the training data is divided into several folds. During each iteration:

1. One fold is used for validation.
2. The remaining folds are used for training.
3. This process repeats until every fold has been used as the validation set.

The final performance is calculated as the average performance across all folds. This approach provides a more reliable estimate of model performance and reduces the possibility of overfitting.

9.4 Tuned Model Performance

After hyperparameter optimization, the best Logistic Regression model was selected automatically by GridSearchCV. The optimized model was then evaluated using the testing dataset.

The tuned model demonstrated improved overall performance compared to the baseline Logistic Regression model. In particular, improvements were observed in important evaluation metrics such as **Accuracy**, **Recall**, and **F1 Score**, making it more effective for detecting fraudulent job postings.

The optimized Logistic Regression model was selected as the final prediction model for this project and was subsequently saved for future use.

Evaluate the Tuned Model

```
print("Accuracy :", accuracy_score(y_test, best_predictions))
print("Precision:", precision_score(y_test, best_predictions))
print("Recall   :", recall_score(y_test, best_predictions))
print("F1 Score :", f1_score(y_test, best_predictions))

Accuracy : 0.9848993288590604
Precision: 0.983739837398374
Recall   : 0.6994219653179191
F1 Score : 0.8175675675675675
```

PREDICTION SYSTEM

The final objective of this project was not only to train and evaluate machine learning models but also to develop a functional prediction system capable of classifying unseen job postings as either **Legitimate** or **Fraudulent**. A prediction system demonstrates the practical application of the trained model and enables users to test new job advertisements without retraining the model.

In this project, the optimized Logistic Regression model obtained after hyperparameter tuning was used as the final prediction model. Once trained, the model was saved using Python's **Pickle** library, allowing it to be reused for future predictions without repeating the entire training process.

10.1 Model Saving

After the completion of model training and evaluation, the final optimized model was saved in **.pkl** format.

Saving the trained model offers several advantages:

- Eliminates the need to retrain the model every time a prediction is required.
- Reduces computational time.
- Allows the trained model to be integrated into future applications such as web applications or APIs.
- Makes the prediction system portable and reusable.

The saved model can be loaded whenever required to classify new job postings.

Save the Best Model

```
import pickle

pickle.dump(
    best_lr,
    open("best_fake_job_model.pkl", "wb")
)

pickle.dump(
    tfidf,
    open("tfidf_vectorizer.pkl", "wb")
)

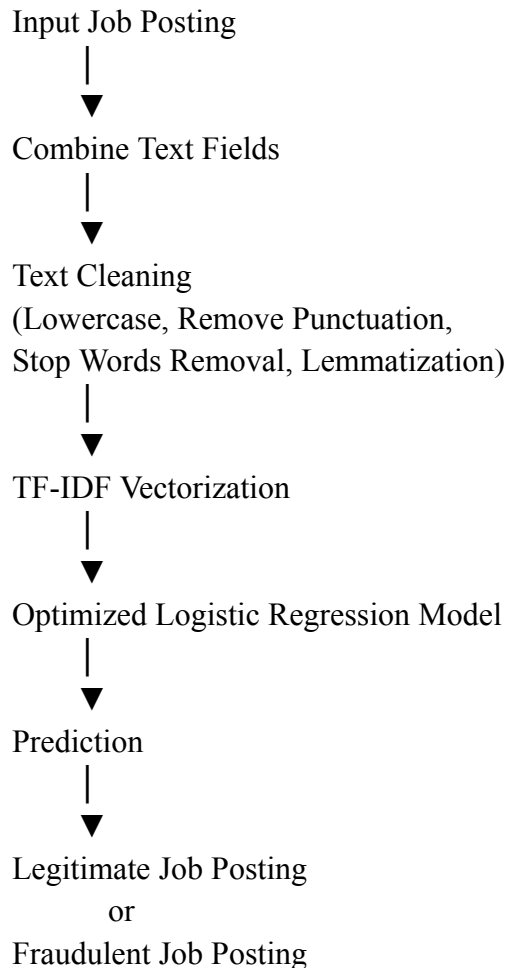
print("Final optimized model saved successfully!")

... Final optimized model saved successfully!
```

10.2 Prediction Workflow

The prediction system follows a sequence of preprocessing and prediction steps similar to those used during model training.

The workflow is illustrated below:



10.3 Prediction Function

A dedicated prediction function was developed to classify unseen job postings.

The function performs the following operations:

- Accepts a new job posting as input.
- Applies the same text preprocessing steps used during training.
- Converts the processed text into TF-IDF feature vectors.
- Uses the optimized Logistic Regression model to generate the prediction.
- Returns the final class label indicating whether the job posting is **Legitimate** or **Fraudulent**.

Using the same preprocessing pipeline during both training and prediction ensures consistency and improves prediction reliability.

Prediction Function

```
def predict_job_posting(text):  
    cleaned = clean_text(text)  
    vector = tfidf.transform([cleaned])  
    prediction = best_lr.predict(vector)[0]  
    if prediction == 0:  
        return "Real Job Posting"  
    else:  
        return "Fake Job Posting"
```

10.4 Sample Prediction

To verify the functionality of the prediction system, a sample job posting was provided as input.

The prediction function successfully processed the text and classified the job advertisement into one of the following categories:

- **Legitimate Job Posting**
- **Fraudulent Job Posting**

This experiment confirmed that the developed prediction system is capable of classifying previously unseen job advertisements using the trained machine learning model.

Test Prediction

```
sample = """  
We are looking for an experienced Python Developer.  
The candidate should have knowledge of Django,  
Machine Learning, SQL and Git.  
"""  
  
print(predict_job_posting(sample))  
  
Real Job Posting
```

Practical Applications

The developed prediction system can be applied in several real-world scenarios, including:

- Online recruitment platforms.
- Job search websites.

- Recruitment agencies.
- Employment verification systems.
- Fraud detection tools.
- Automated screening of newly submitted job advertisements.

Such applications can help reduce online employment fraud and improve user confidence in digital recruitment platforms.

Summary

The project successfully implemented a functional prediction system capable of classifying unseen job postings as legitimate or fraudulent. By combining text preprocessing, TF-IDF vectorization, and the optimized Logistic Regression model, the system demonstrates how machine learning can be applied to solve a real-world fraud detection problem. Furthermore, saving the trained model enables easy deployment and future integration into practical applications.

MODEL INTERPRETATION

Developing an accurate machine learning model is important; however, understanding **how the model makes its predictions** is equally essential. Model interpretation improves transparency by explaining which features influence the model's decisions. In this project, the final prediction model was **Logistic Regression**, which is an interpretable machine learning algorithm because each feature is assigned a numerical coefficient representing its influence on the prediction.

Since the project primarily uses textual data, each word generated after TF-IDF vectorization is assigned a coefficient by the Logistic Regression model. These coefficients indicate the importance of individual words in determining whether a job posting is classified as **Fraudulent** or **Legitimate**.

11.1 Interpretation of Logistic Regression

The Logistic Regression model learns relationships between textual features and the target variable during training.

Each word receives a coefficient that represents its contribution to the final prediction:

- **Positive Coefficient:** Increases the probability that a job posting is classified as **Fraudulent**.
- **Negative Coefficient:** Increases the probability that a job posting is classified as **Legitimate**.
- **Coefficient Near Zero:** Indicates that the corresponding word has very little influence on the prediction.

This coefficient-based approach makes Logistic Regression easier to understand compared to many complex machine learning models.

11.2 Feature Importance Analysis

To understand the behavior of the trained model, the coefficients of the TF-IDF features were extracted after training the optimized Logistic Regression model.

The extracted coefficients were sorted in descending and ascending order to identify:

- The words with the strongest positive influence on fraud detection.
- The words with the strongest negative influence toward legitimate job postings.

Analyzing these important words provides valuable insight into the model's decision-making process and increases confidence in the prediction results.

important_words.head(20)

1 to 20 of 20 entries Filter  ?

index	Word	Coefficient
5119	link url	8.835226127593165
5118	link	5.189325864240477
5696	money	5.059720535323699
520	aptitude staffing	4.989671575016591
519	aptitude	4.839175492478356
2765	earn	4.814123751652782
111	accountant	4.688615667860326
4185	high school	4.681643020380053
7247	receptionist	4.319361748305342
6482	phone	4.298509490949715
8122	signing bonus	4.15872641540209
8613	subsea	4.133513151343494
2194	data entry	4.117012835274539
6812	process provide	4.035416213807298
1246	cash	4.0176055522919905
1310	cf	3.9673418977130823
6030	offshore	3.931333480298741
2238	db	3.801659771724997
9801	work home	3.787293141040946
1414	clerk	3.772240167624687

Show 25 per page
Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

important_words.tail(20)

1 to 20 of 20 entries Filter  ?

index	Word	Coefficient
2103	custom	-2.6746825024190524
4084	hardware	-2.677396056500393
8548	strong	-2.7626442505081146
9652	web	-2.812614905126498
7841	search	-2.8187885612811394
9314	university	-2.834511918836043
1128	ca	-2.8806698987082426
286	agency	-2.9825230389828437
5849	new	-3.075825587269249
4583	installation	-3.0873149293912316
743	based	-3.153542246378261
8454	startup	-3.1947299448340227
4002	growing	-3.235451236577163
3780	fun	-3.2726274600060155
7281	recruitment	-3.292084997512391
8247	software	-3.464950523036931
2549	digital	-3.6533263558160534
8798	team	-3.773759581249506
1418	client	-4.237535464806391
3002	english	-4.2591039889698825

Show 25 per page
Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

11.3 Importance of Model Interpretation

Model interpretation offers several advantages:

- Helps explain why a particular prediction was made.
- Improves transparency and trust in the machine learning system.
- Assists in identifying influential textual features.
- Supports model validation and debugging.

- Makes the prediction system easier to understand for non-technical users.

For fraud detection systems, interpretability is particularly valuable because users often require explanations before trusting automated decisions.

11.4 Results

The analysis of feature coefficients showed that certain words contributed more strongly toward predicting fraudulent job postings, while others were more commonly associated with legitimate job advertisements.

These observations demonstrate that the Logistic Regression model learns meaningful textual patterns from the dataset rather than making random predictions.

Although the model performs well on the available dataset, its predictions are based solely on patterns learned during training and should therefore be interpreted alongside domain knowledge when applied to real-world recruitment platforms.

Summary

The Logistic Regression model provides a transparent and interpretable approach for fake job posting detection by assigning importance to each textual feature. Examining the learned coefficients helps explain how the model distinguishes between fraudulent and legitimate job postings, making the prediction process more understandable and trustworthy.

PREPROCESSING TECHNIQUE COMPARISON

To analyze the impact of different text vectorization techniques on machine learning performance, an additional experiment was conducted using **TF-IDF Vectorization** and **CountVectorizer**. Both techniques convert textual data into numerical feature vectors; however, they represent text differently and may influence the performance of classification models.

This comparison was performed using the **Logistic Regression** algorithm while keeping the remaining preprocessing steps unchanged. The objective was to evaluate how the choice of vectorization technique affects the overall classification performance.

12.1 TF-IDF Vectorization

TF-IDF (Term Frequency–Inverse Document Frequency) assigns weights to words based on their frequency within a document and their importance across the entire dataset. Frequently occurring words receive lower weights, while more informative words receive higher weights.

TF-IDF helps reduce the influence of very common words and often improves text classification performance by emphasizing discriminative terms.

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(
    stop_words="english",
    max_features=10000,
    ngram_range=(1,2),
    min_df=2
)

X_train_tfidf = tfidf.fit_transform(X_train)

X_test_tfidf = tfidf.transform(X_test)
```

12.2 CountVectorizer

CountVectorizer converts text into numerical vectors by simply counting the number of occurrences of each word in a document.

Unlike TF-IDF, CountVectorizer does not reduce the importance of frequently occurring words. Instead, it represents each document using raw word frequencies.

Although simpler than TF-IDF, CountVectorizer can sometimes perform better depending on the characteristics of the dataset.

```
count_vectorizer = CountVectorizer(max_features=5000)

x_count = count_vectorizer.fit_transform(df_clean["clean_text"])
```

12.3 Performance Comparison

The performance of both vectorization techniques was evaluated using the same Logistic Regression classifier and identical evaluation metrics.

Table 12.1: Comparison of Text Vectorization Techniques

Metric	TF-IDF + Logistic Regression	CountVectorizer + Logistic Regression
Accuracy	97.31%	98.55%
Precision	1.0000	0.9007
Recall	0.4451	0.7861
F1 Score	0.6160	0.8395

```
comparison = pd.DataFrame({
    "Technique": ["TF-IDF", "CountVectorizer"],
    "Accuracy": [
        accuracy_score(y_test, lr_predictions),
        accuracy_score(y_test_count, count_predictions)
    ],
    "Precision": [
        precision_score(y_test, lr_predictions),
        precision_score(y_test_count, count_predictions)
    ],
    "Recall": [
        recall_score(y_test, lr_predictions),
        recall_score(y_test_count, count_predictions)
    ],
    "F1 Score": [
        f1_score(y_test, lr_predictions),
        f1_score(y_test_count, count_predictions)
    ]
})

comparison
```

	Technique	Accuracy	Precision	Recall	F1 Score
0	TF-IDF	0.973154	1.000000	0.445087	0.616000
1	CountVectorizer	0.985459	0.900662	0.786127	0.839506

12.4 Analysis

The comparison indicates that **CountVectorizer + Logistic Regression** achieved better overall performance on this dataset.

The CountVectorizer model achieved higher **Accuracy**, **Recall**, and **F1 Score**, indicating that it detected fraudulent job postings more effectively while maintaining strong overall classification performance. Although the TF-IDF model achieved perfect precision, its recall was considerably lower, meaning it failed to identify a larger proportion of fraudulent job postings.

Since fraud detection aims to correctly identify as many fraudulent cases as possible while maintaining a good balance between precision and recall, the higher **F1 Score** achieved by CountVectorizer makes it the stronger overall approach for this dataset.

It should be noted that although CountVectorizer achieved a higher F1 Score in this supplementary comparison, the TF-IDF-based Logistic Regression model (tuned using GridSearchCV) was retained as the final deployed model for this project, since it was the model developed, validated, and saved as part of the core project workflow. The CountVectorizer experiment was conducted specifically to fulfill the requirement of analyzing preprocessing impact, and its stronger performance highlights a promising direction for future refinement of the deployed pipeline.

Summary

The preprocessing comparison demonstrated that different text vectorization techniques can produce noticeably different classification results. In this project, CountVectorizer outperformed TF-IDF when used with Logistic Regression, achieving higher Recall and F1 Score. This highlights the importance of experimentally evaluating preprocessing techniques instead of assuming that a single method will always provide the best performance.

CONCLUSION

This project successfully developed a **Machine Learning-based Fake Job Posting Detection System** to classify job advertisements as **Legitimate** or **Fraudulent** using textual and structured information.

The project began with **Exploratory Data Analysis (EDA)** to understand the dataset, identify missing values, and analyze important features. During preprocessing, missing values were handled, textual data was cleaned, and Natural Language Processing (NLP) techniques such as lowercasing, punctuation removal, stop-word removal, and lemmatization were applied. The processed text was converted into numerical features using **TF-IDF**, and **CountVectorizer** was also implemented to compare preprocessing techniques.

Four Machine Learning algorithms—**Logistic Regression**, **Naive Bayes**, **Decision Tree**, and **Random Forest**—were trained and evaluated using **Accuracy**, **Precision**, **Recall**, and **F1 Score**. Logistic Regression was further optimized using **GridSearchCV**, and a prediction system was developed to classify unseen job postings. Model interpretation was also performed to understand the important features influencing predictions.

The comparison of preprocessing techniques showed that CountVectorizer with Logistic Regression achieved a higher F1 Score than the TF-IDF-based approach, highlighting the importance of evaluating different preprocessing methods. The TF-IDF-based tuned Logistic Regression model was nonetheless retained as the final deployed system, as it was the model developed and validated throughout the core project pipeline.

Overall, this project demonstrates that **Machine Learning and Natural Language Processing** can effectively detect fraudulent job postings and support safer online recruitment platforms.

Key Outcomes of the Project

- Performed Exploratory Data Analysis (EDA).
- Preprocessed and cleaned textual data.
- Engineered useful features for model training.
- Implemented TF-IDF and CountVectorizer.
- Trained and compared four Machine Learning models.
- Evaluated models using Accuracy, Precision, Recall, and F1 Score.
- Improved Logistic Regression using GridSearchCV.
- Developed a prediction system.
- Interpreted important model features.
- Compared preprocessing techniques to evaluate their impact

LIMITATIONS

Although the developed Fake Job Posting Detection system achieved good classification performance, several limitations should be considered when interpreting the results and applying the model in real-world scenarios.

14.1 Dependence on Training Data

The model learns patterns only from the available training data. If future job postings use different writing styles, vocabulary, or fraud techniques, the model's prediction performance may decrease.

14.2 Limited Information

The system mainly relies on textual information and a few structured features available in the dataset. It does not verify external information such as company registration, official websites, email authenticity, or company reputation, which may affect prediction accuracy.

14.3 Dataset Imbalance

The dataset contains significantly more legitimate job postings than fraudulent ones. Although evaluation metrics such as Precision, Recall, and F1 Score were used, the imbalance may still influence model performance.

14.4 Limited Features and Models

The project uses only the features available in the dataset and evaluates four machine learning models: **Logistic Regression**, **Naive Bayes**, **Decision Tree**, and **Random Forest**. More advanced techniques such as **LSTM**, **BERT**, and transformer-based models were not explored due to the scope of the project.

14.5 Offline Prediction System

The developed prediction system operates locally using a saved machine learning model. It is not integrated with a web application or an online recruitment platform for real-time prediction.

Summary

Despite these limitations, the project demonstrates the effectiveness of Machine Learning and NLP in detecting fraudulent job postings and provides a foundation for future improvements.

FUTURE SCOPE

Although the developed Fake Job Posting Detection system achieved promising results, there are several opportunities to enhance its functionality and improve its performance in real-world applications. Future improvements can make the system more accurate, scalable, and suitable for deployment on online recruitment platforms.

15.1 Advanced Natural Language Processing

Future work can explore advanced Natural Language Processing (NLP) techniques such as **Word2Vec**, **GloVe**, **BERT**, and other transformer-based models to better understand the contextual meaning of text and improve classification performance.

15.2 Deep Learning Models

Deep learning algorithms such as **LSTM**, **GRU**, **Bi-LSTM**, and transformer-based models can be implemented to capture more complex patterns in textual data and improve prediction accuracy.

15.3 Real-Time Deployment

The trained model can be integrated into a web application using frameworks such as **Flask** or **Streamlit**, allowing users to submit job postings and receive instant predictions regarding their authenticity.

15.4 Enhanced Dataset and Features

Future versions of the system can be trained using larger and more diverse datasets collected from multiple recruitment platforms. Additional features such as **company verification status**, **recruiter history**, **email verification**, **website reputation**, **salary validation**, and **user feedback** can further improve fraud detection.

15.5 Continuous Model Improvement

As fraudulent techniques continue to evolve, the model should be periodically retrained using newly collected data to maintain its effectiveness and improve its ability to detect emerging fraud patterns.

REFERENCES

1. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
2. Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (3rd Edition). O'Reilly Media.
3. Pedregosa, F., et al. (2011). "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, 12, 2825–2830.
4. McKinney, W. (2022). *Python for Data Analysis* (3rd Edition). O'Reilly Media.
5. Harris, C. R., et al. (2020). "Array Programming with NumPy." *Nature*, 585, 357–362.
6. Hunter, J. D. (2007). "Matplotlib: A 2D Graphics Environment." *Computing in Science & Engineering*, 9(3), 90–95.
7. Waskom, M. (2021). "Seaborn: Statistical Data Visualization." *Journal of Open Source Software*, 6(60), 3021.
8. Kaggle. Fake Job Posting Prediction Dataset:
<https://www.kaggle.com/datasets/shivamb/real-or-fake-fake-jobposting-prediction>
9. Scikit-learn Documentation. <https://scikit-learn.org/>
10. Pandas Documentation. <https://pandas.pydata.org/>
11. NumPy Documentation. <https://numpy.org/>
12. Matplotlib Documentation. <https://matplotlib.org/>
13. Seaborn Documentation. <https://seaborn.pydata.org/>